

Egypt University of Informatics
Faculty of Computing and Information Sciences
Computer Vision Course

CV-2: Crop Disease Detection

A Transfer Learning System for Plant Leaf Disease Classification

Project Information

Course: Computer Vision
Project Code: CV-2
Project Topic: Crop Disease Detection
Framework: PyTorch
Model: EfficientNet-B0 with Transfer Learning
Number of Classes: 38 crop and disease categories
GitHub Repository: <https://github.com/ziadabdelhafizz/crop-disease-detection>

Done By

Name	ID
Ziad Abdelhafiz	22-101236
Yehia Mahrous	22-101239
Seif Hegazy	22-101049

Under the Supervision of

Dr. Mona Soliman
Dr. Mennatullah Hassan

Academic Year 2025–2026

May 18, 2026

Contents

Abstract	1
1 Introduction	1
2 Problem Statement and Objectives	2
2.1 Problem Statement	2
2.2 Project Objectives	2
2.3 Scope	3
3 Related Work	3
3.1 Deep Learning for Plant Disease Detection	3
3.2 Transfer Learning	4
3.3 EfficientNet	4
3.4 Explainability in Vision Models	4
4 Dataset Description	4
4.1 Dataset Source	4
4.2 Dataset Classes	5
4.3 Train, Validation, and Test Split	5
4.4 Dataset Strengths and Limitations	5
5 Methodology	6
5.1 System Architecture	6
5.2 Preprocessing	6
5.3 Data Augmentation	6
5.4 Loss Function	7
5.5 Optimizer and Learning Strategy	7
6 Model Architecture	7
6.1 EfficientNet-B0 Backbone	7
6.2 Classifier Head	8
6.3 Implementation Details	8
6.4 Why EfficientNet-B0 Was Selected	8
7 Experimental Setup	9
7.1 Development Environment	9
7.2 Training Configuration	9
7.3 Evaluation Metrics	9

8	Results	10
8.1	Training and Validation Performance	10
8.2	Final Test Results	11
8.3	Confusion Matrix	11
8.4	Top Error Cases	12
9	Error Analysis and Discussion	13
9.1	Why Similar Diseases Are Confused	13
9.2	Crop Leakage and Background Bias	13
9.3	Confidence Scores	14
9.4	Training vs. Validation Accuracy Behavior	14
10	Application and Demo	14
10.1	Command-Line Inference	14
10.2	Gradio Web Demo	15
11	Repository Structure and Reproducibility	15
11.1	Repository Structure	15
11.2	Reproducible Instructions	16
11.3	Role of Each Source File	17
12	Limitations	17
13	Future Work	17
14	Conclusion	18
A	Appendix A: Full Class List	18
B	Appendix B: Key Implementation Choices	20
B.1	Why the Dataset Was Not Pushed Fully to GitHub	20
B.2	Why Top-3 Predictions Were Added	20
B.3	Why Gradio Was Used	20
C	Appendix C: Important Commands	20
C.1	Training Command	20
C.2	Evaluation Command	21
C.3	Prediction Command	21
C.4	Demo Command	21

List of Figures

1	Dataset split verification from the project notebook.	5
2	High-level architecture of the crop disease detection system.	6
3	Training and validation loss curves over 5 epochs.	10
4	Training and validation accuracy curves over 5 epochs.	11
5	Normalized confusion matrix for the 38-class test set.	12
6	Working Gradio demo predicting Corn (maize) Common rust with top-3 probabilities.	15

List of Tables

1	Final dataset split used in this project.	5
2	Training configuration.	9
3	Final test performance.	11
4	Top confusion cases on the test set.	12
5	Main source files and their responsibilities.	17

Abstract

Crop diseases can reduce agricultural productivity and create economic losses when infections are not detected early. Manual disease inspection depends on expert availability, crop type, lighting conditions, and field experience. This project presents a functional computer vision system for classifying plant leaf images into crop disease categories. The system uses transfer learning with EfficientNet-B0, pretrained on ImageNet, and fine-tunes a custom classification head for 38 plant health categories. The dataset used in this project is the New Plant Diseases Dataset from Kaggle, a PlantVillage-style dataset containing approximately 87,000 RGB leaf images across 38 classes. The final processed split contains 70,295 training images, 8,777 validation images, and 8,795 test images.

The implemented pipeline includes dataset preparation, preprocessing, augmentation, model training, validation, testing, single-image inference, visual evaluation, a Gradio web demo, and a documented GitHub repository. The model achieved a test accuracy of 94.59%, weighted precision of 94.70%, weighted recall of 94.59%, and weighted F1-score of 94.58% on the held-out test set. A confusion matrix and error analysis show that most mistakes occur between visually similar diseases, especially diseases affecting the same crop species. The final system is reproducible, documented, and suitable for a university-level computer vision project. The main limitation is that the dataset consists mostly of controlled leaf images, so future work should include field-image training, leaf segmentation, and explainability methods such as Grad-CAM.

1 Introduction

Agriculture is one of the most important sectors for food security and economic stability. Crop diseases caused by fungi, bacteria, viruses, pests, and environmental stress can significantly reduce yield and quality. Early detection is therefore essential because treatment is often more effective before the disease spreads across a field. Traditional disease identification relies on visual inspection by farmers, agronomists, or plant pathologists. Although expert inspection can be accurate, it is not always scalable, especially when expert access is limited, the farm area is large, or symptoms develop rapidly.

Computer vision offers a practical approach for supporting automated plant disease diagnosis. When leaf symptoms such as spots, lesions, discoloration, mildew, or blight are visible in images, convolutional neural networks (CNNs) can learn discriminative visual patterns. CNNs have become highly effective for image classification because they learn spatial features directly from pixel data instead of depending on manually engineered descriptors. In agricultural applications, this means a model can learn the difference between healthy leaves and diseased leaves, and in some cases distinguish between multiple diseases on the same crop.

The goal of this project is to build a complete crop disease detection system, not only a model. The final system includes data processing, model development, evaluation, error analysis, and deployment through a simple web interface. This makes the project aligned with the practical requirements of a computer vision course: coding, functionality, accuracy, documentation, and reproducibility. The chosen model is EfficientNet-B0 because it provides a strong balance between accuracy and computational efficiency. It is lightweight enough to train in Google Colab while still being accurate enough for a 38-class classification problem.

This report is organized as follows. Section 2 defines the problem statement and objectives. Section 3 discusses related work. Section 4 describes the dataset. Section 5 explains the methodology and system architecture. Section 6 details the model architecture and training procedure. Section 7 presents the experiments and results. Section 8 analyzes errors and limitations. Section 9 explains the application demo and repository structure. Finally, Section 10 concludes the project and proposes future improvements.

2 Problem Statement and Objectives

2.1 Problem Statement

The project addresses the problem of identifying crop diseases from plant leaf images. Given an input image of a leaf, the system must predict the most likely crop-disease category. Each category is a combination of a crop species and a health condition. For example, a tomato leaf may be classified as healthy, early blight, late blight, bacterial spot, leaf mold, septoria leaf spot, target spot, spider mites, mosaic virus, or yellow leaf curl virus.

Formally, the task is a supervised multi-class image classification problem. Let x_i be an input RGB image and y_i be its corresponding label from a set of $C = 38$ classes. The model learns a function:

$$f_{\theta}(x_i) \rightarrow \hat{y}_i, \quad \hat{y}_i \in \{1, 2, \dots, 38\} \quad (1)$$

where θ represents the trainable model parameters. The objective is to minimize classification error on unseen validation and test images.

2.2 Project Objectives

The project objectives are listed below:

1. Build a reproducible crop disease detection pipeline using PyTorch.

2. Use transfer learning with a strong CNN backbone to improve accuracy and reduce training time.
3. Train a model capable of classifying 38 crop-disease categories.
4. Evaluate the model using standard classification metrics: accuracy, precision, recall, F1-score, confusion matrix, and inference examples.
5. Create a simple inference script for single-image prediction.
6. Create a Gradio demo that allows users to upload a plant leaf image and receive top-3 predictions.
7. Document the full project in a GitHub repository with installation, dataset, training, evaluation, and demo instructions.

2.3 Scope

The scope of this project is image-level classification. It does not attempt to segment the diseased region, estimate disease severity, or provide agricultural treatment recommendations. The model predicts disease categories based on visible leaf symptoms. The system is designed as a prototype for academic purposes and should not be used as a replacement for expert agricultural diagnosis in real farms without additional validation.

3 Related Work

3.1 Deep Learning for Plant Disease Detection

Plant disease detection using deep learning became widely studied after the release of open leaf-image datasets such as PlantVillage. Hughes and Salathé introduced an open image repository to support mobile disease diagnostics [1]. Later, Mohanty et al. demonstrated that deep CNNs can classify plant diseases with high accuracy using the PlantVillage dataset [2]. Their work showed that CNNs can learn disease-specific visual patterns such as spots, discoloration, texture changes, and leaf deformation.

Earlier machine learning approaches often depended on handcrafted features such as color histograms, texture descriptors, edge information, and shape features. Although useful in constrained settings, handcrafted features are sensitive to lighting, background, and symptom variation. CNNs reduce the need for manual feature engineering by learning hierarchical features automatically. Lower convolutional layers typically learn edges, color transitions, and simple textures, while deeper layers learn more specific patterns such as lesions, mildew regions, or disease-like color distributions.

3.2 Transfer Learning

Training a CNN from scratch requires large datasets and substantial computational resources. Transfer learning addresses this problem by starting from a model pretrained on a large dataset such as ImageNet [3]. The pretrained model already contains general visual representations that can be adapted to a new task. In this project, the feature extractor of EfficientNet-B0 is initialized with ImageNet pretrained weights, and the final classifier is replaced with a new 38-class classifier. This approach reduces training time and improves generalization compared to training all parameters from random initialization.

3.3 EfficientNet

EfficientNet was proposed by Tan and Le as a family of CNN architectures based on compound scaling [4]. Instead of scaling depth, width, or resolution independently, EfficientNet scales them together in a balanced way. This produces models that achieve strong accuracy while remaining computationally efficient. EfficientNet-B0 is the baseline member of the family and is suitable for this project because it is accurate enough for multi-class plant disease classification while remaining lightweight enough for Google Colab training and demo deployment.

3.4 Explainability in Vision Models

A common limitation of CNN-based classifiers is that their predictions are not directly interpretable. Explainability methods such as Grad-CAM can highlight image regions that contribute most strongly to a prediction [5]. Although Grad-CAM is not required for the core functionality, it is a valuable future addition because it can show whether the model focuses on diseased leaf regions or irrelevant background artifacts.

4 Dataset Description

4.1 Dataset Source

The dataset used in this project is the New Plant Diseases Dataset from Kaggle. The dataset page describes it as containing about 87K RGB images of healthy and diseased crop leaves categorized into 38 different classes [6]. The task is closely related to PlantVillage, which TensorFlow Datasets describes as 54,303 healthy and unhealthy leaf images divided into 38 categories by species and disease [7]. The selected dataset is appropriate because it directly matches the project requirement: identifying plant diseases from images.

4.2 Dataset Classes

The project uses all 38 classes. Each class name follows the structure:

`Crop_name___Disease_name`

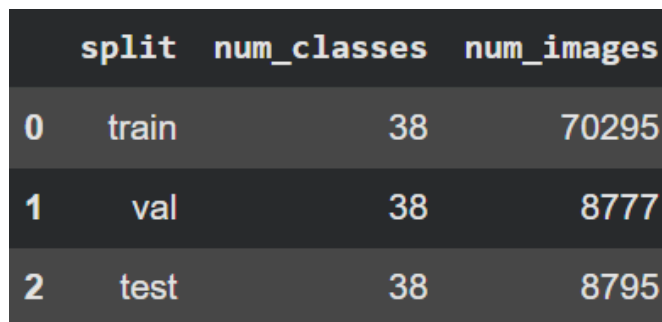
Examples include `Apple___Apple_scab`, `Corn_(maize)___Common_rust_`, `Potato___Late_blight`, and `Tomato___healthy`. The class list contains both healthy and diseased examples, allowing the model to identify whether a leaf is healthy or affected by a specific disease.

4.3 Train, Validation, and Test Split

The dataset was organized into training, validation, and test splits. The original dataset already contained training and validation folders. For stronger evaluation, the validation set was split into validation and test subsets. The final split is shown in Table 1.

Table 1: Final dataset split used in this project.

Split	Number of Classes	Number of Images
Training	38	70,295
Validation	38	8,777
Test	38	8,795



A screenshot of a Jupyter Notebook table with a dark background. The table has four columns: an index column, a 'split' column, a 'num_classes' column, and a 'num_images' column. The data is as follows:

	split	num_classes	num_images
0	train	38	70295
1	val	38	8777
2	test	38	8795

Figure 1: Dataset split verification from the project notebook.

4.4 Dataset Strengths and Limitations

The main strength of the dataset is its size and class coverage. With more than 70,000 training images and 38 classes, the dataset is large enough to train a reliable deep learning classifier. Another strength is that the classes are well organized in folders, making it compatible with PyTorch’s `ImageFolder` dataset loader.

The main limitation is that many images are captured in controlled conditions, often with a plain or consistent background. This can make the task easier than real-world field diagnosis. A model trained only on controlled images may perform worse on images taken

outdoors with complex backgrounds, shadows, occlusion, different cameras, or multiple leaves. This limitation is addressed partially through data augmentation, but future work should include field-image datasets and domain adaptation.

5 Methodology

5.1 System Architecture

The final system follows a modular architecture. The input image is first resized and normalized. During training, augmentation is applied to increase robustness. The image is then passed through an EfficientNet-B0 model. The model outputs logits for 38 classes, which are converted into probabilities using the softmax function. The final prediction is the class with the highest probability. For the demo, the system also returns the top-3 predicted classes.

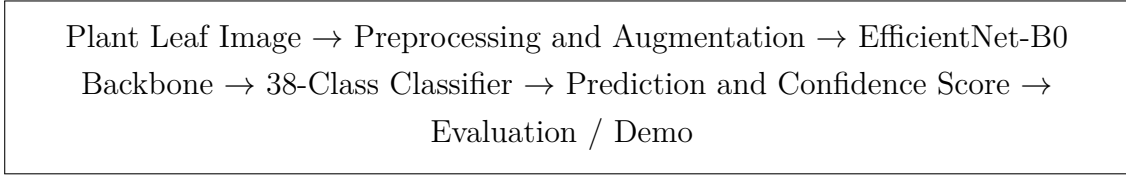


Figure 2: High-level architecture of the crop disease detection system.

5.2 Preprocessing

All images are converted to RGB and resized to match the input resolution expected by EfficientNet-B0. The model uses an input size of 224×224 . Images are normalized using ImageNet mean and standard deviation values because the backbone is pretrained on ImageNet. The normalization is defined as:

$$x' = \frac{x - \mu}{\sigma} \quad (2)$$

where x is the image tensor, $\mu = [0.485, 0.456, 0.406]$, and $\sigma = [0.229, 0.224, 0.225]$.

5.3 Data Augmentation

Data augmentation is applied only to the training set. The validation and test sets use deterministic resizing, center cropping, and normalization. The training augmentations include:

- Random resized crop to simulate scale and framing variation.
- Horizontal flip to account for orientation changes.

- Vertical flip with lower probability.
- Random rotation up to 20 degrees.
- Color jitter for brightness, contrast, saturation, and hue variation.

These augmentations make the training task harder, which explains why training accuracy can be lower than validation accuracy. This is not necessarily a sign of poor learning; it often indicates that the model is being trained on more difficult transformed images while validation uses clean deterministic images.

5.4 Loss Function

The model is trained using cross-entropy loss. Label smoothing is applied with a smoothing factor of 0.05. Label smoothing reduces overconfidence by preventing the model from assigning probability 1.0 to the target class during training. This can improve generalization in multi-class classification tasks.

For a sample with true class y , predicted probability p_y , and C total classes, cross-entropy loss is:

$$\mathcal{L} = - \sum_{c=1}^C y_c \log(p_c) \quad (3)$$

where y_c is the target distribution. With label smoothing, the target distribution is softened rather than being a strict one-hot vector.

5.5 Optimizer and Learning Strategy

The optimizer used is AdamW, which is an improved version of Adam with decoupled weight decay. Adam is widely used for deep learning optimization [8]. Weight decay is set to 10^{-4} to reduce overfitting. A ReduceLROnPlateau scheduler monitors validation accuracy and decreases the learning rate when performance stops improving. Early stopping is also implemented to stop training if validation accuracy does not improve for a fixed number of epochs.

6 Model Architecture

6.1 EfficientNet-B0 Backbone

EfficientNet-B0 is used as the feature extractor. The model is loaded with pretrained ImageNet weights. The final classification layer of the original model is replaced with a new classifier for 38 output classes. The project initially freezes the backbone and trains the final classifier head. This is a common transfer learning strategy because it allows the

new classification layer to learn task-specific decision boundaries while preserving useful pretrained visual features.

6.2 Classifier Head

The original EfficientNet classifier is replaced by:

- Dropout layer with probability 0.3.
- Fully connected linear layer with 38 output units.

Dropout helps reduce overfitting by randomly dropping activations during training. The final linear layer maps the EfficientNet feature representation to the 38 crop disease classes.

6.3 Implementation Details

The model is implemented in `src/model.py`. A simplified version of the model construction logic is shown below.

Listing 1: Simplified EfficientNet-B0 model creation.

```
from torchvision import models
import torch.nn as nn

def create_model(num_classes, freeze_backbone=True):
    weights = models.EfficientNet_B0_Weights.DEFAULT
    model = models.efficientnet_b0(weights=weights)

    if freeze_backbone:
        for param in model.features.parameters():
            param.requires_grad = False

    in_features = model.classifier[1].in_features
    model.classifier = nn.Sequential(
        nn.Dropout(p=0.3),
        nn.Linear(in_features, num_classes)
    )
    return model
```

6.4 Why EfficientNet-B0 Was Selected

EfficientNet-B0 was selected for three main reasons. First, it provides strong accuracy compared to its computational size. Second, it is supported directly in `torchvision`,

making the project easier to reproduce. Third, it is practical for Google Colab because it can train with a batch size of 32 and an image size of 224 without requiring excessive GPU memory.

7 Experimental Setup

7.1 Development Environment

The project was developed and trained in Google Colab using a GPU runtime. PyTorch was used as the deep learning framework. The repository was organized professionally with separate files for dataset loading, model definition, training, evaluation, prediction, and the Gradio application. The final GitHub repository is:

<https://github.com/ziadabdelhafizz/crop-disease-detection>

7.2 Training Configuration

The baseline training configuration is summarized in Table 2.

Table 2: Training configuration.

Parameter	Value
Model	EfficientNet-B0
Pretraining	ImageNet pretrained weights
Input size	224×224
Number of classes	38
Batch size	32
Epochs	5
Optimizer	AdamW
Initial learning rate	0.001
Weight decay	0.0001
Loss function	Cross-entropy loss
Label smoothing	0.05
Early stopping patience	3
Learning-rate scheduler	ReduceLROnPlateau

7.3 Evaluation Metrics

The model is evaluated using accuracy, weighted precision, weighted recall, weighted F1-score, loss, confusion matrix, and sample predictions. Accuracy measures the percentage

of correctly classified images:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

For multi-class classification, precision, recall, and F1-score are computed per class and then averaged using class support weights. Weighted averaging is appropriate because it accounts for the number of samples in each class.

8 Results

8.1 Training and Validation Performance

The training and validation curves show that the model learns quickly within the first few epochs. The validation accuracy reaches 94.49% after 5 epochs, while training accuracy reaches 89.76%. The validation accuracy is higher than the training accuracy because the training data is augmented and regularized through dropout and label smoothing. Therefore, the training set is effectively more difficult than the validation set.

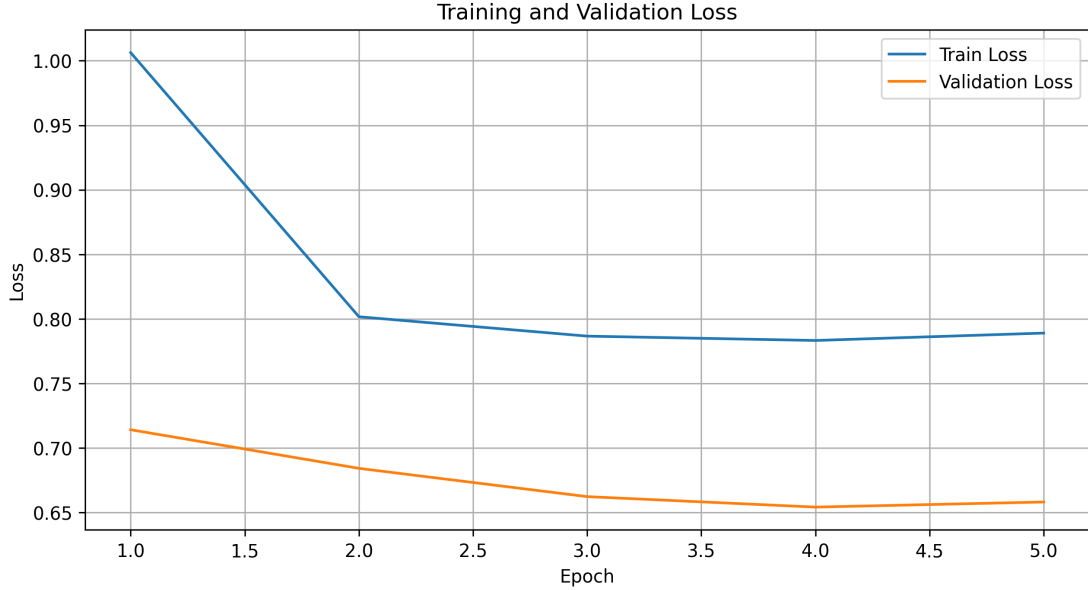


Figure 3: Training and validation loss curves over 5 epochs.

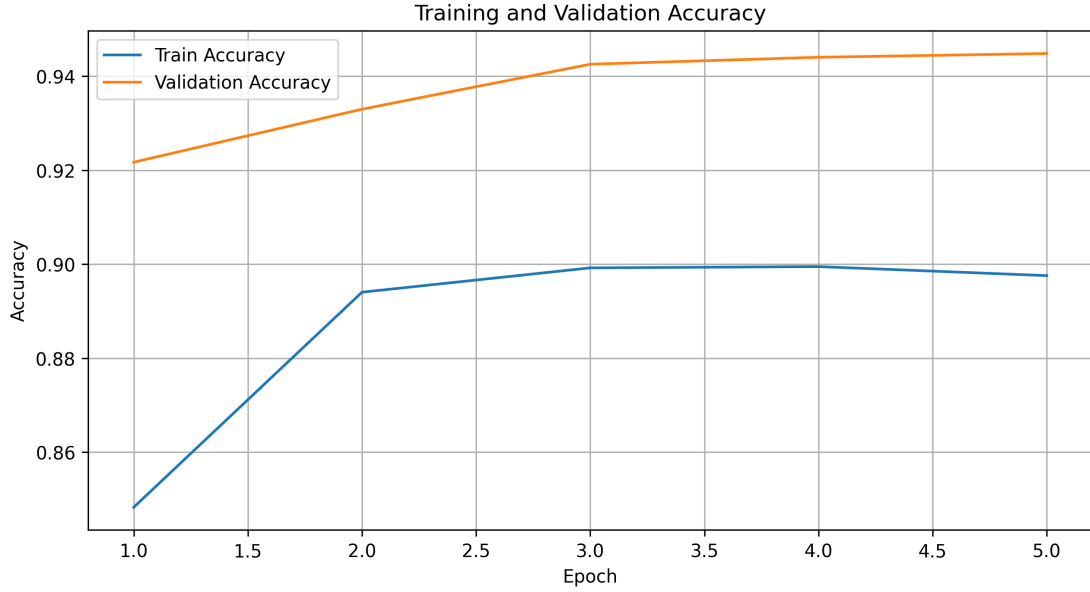


Figure 4: Training and validation accuracy curves over 5 epochs.

8.2 Final Test Results

The model was evaluated on the held-out test set containing 8,795 images. The final results are shown in Table 3. These results are not placeholders; they are the actual measured outputs from the implemented evaluation script.

Table 3: Final test performance.

Metric	Value
Test loss	0.2401
Test accuracy	94.59%
Weighted precision	94.70%
Weighted recall	94.59%
Weighted F1-score	94.58%
Number of test images	8,795
Number of classes	38

8.3 Confusion Matrix

The normalized confusion matrix in Figure 5 shows that most predictions fall along the diagonal, indicating strong classification performance. Off-diagonal values correspond to misclassifications. The strongest confusions occur between diseases that affect the same crop or have visually similar symptoms.

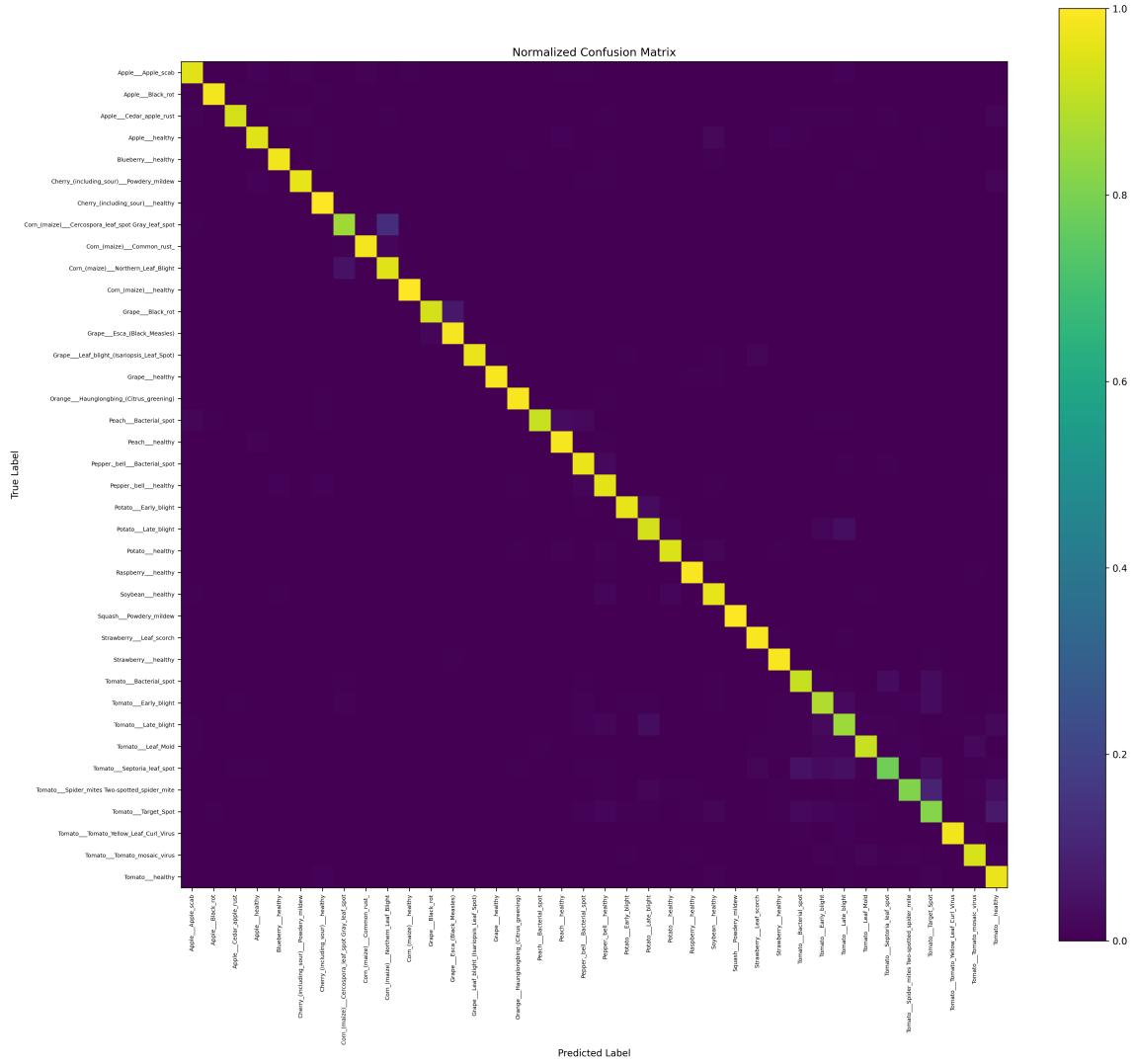


Figure 5: Normalized confusion matrix for the 38-class test set.

8.4 Top Error Cases

Table 4 lists the most common wrong predictions. The largest confusion is between Corn_(maize)__Cercospora_leaf_spot Gray_leaf_spot and Corn_(maize)__Northern_Leaf_Blight. This is reasonable because both are corn leaf diseases and can produce elongated or spotted lesions. Similar confusion also occurs among tomato leaf diseases, where target spot, septoria leaf spot, bacterial spot, and late blight may share color and texture patterns.

Table 4: Top confusion cases on the test set.

True Class	Predicted Class	Count
Corn Cercospora leaf spot / Gray leaf spot	Corn Northern Leaf Blight	27
Tomato Spider mites	Tomato Target Spot	22

True Class	Predicted Class	Count
Tomato Target Spot	Tomato healthy	16
Grape Black rot	Grape Esca / Black Measles	15
Corn Northern Leaf Blight	Corn Cercospora leaf spot / Gray leaf spot	12
Tomato Septoria leaf spot	Tomato Bacterial spot	11
Potato Late blight	Tomato Late blight	9
Tomato Septoria leaf spot	Tomato Late blight	9
Tomato Late blight	Potato Late blight	8
Tomato Septoria leaf spot	Tomato Target Spot	8
Tomato Spider mites	Tomato healthy	8
Peach Bacterial spot	Peach healthy	7
Potato Early blight	Potato Late blight	7
Tomato Early blight	Tomato Target Spot	7
Tomato Bacterial spot	Tomato Septoria leaf spot	6

9 Error Analysis and Discussion

9.1 Why Similar Diseases Are Confused

Most errors occur between diseases of the same crop or visually similar diseases across crops. This is expected because leaf disease symptoms are often not unique. Multiple diseases can produce brown lesions, yellowing, dark spots, or necrotic areas. For example, tomato target spot and tomato spider mite damage can both appear as spotted leaf discoloration. Similarly, early blight and late blight can both create dark damaged regions, although their biological causes and visual progression differ.

9.2 Crop Leakage and Background Bias

Although the model performs well, controlled datasets can introduce background and crop-specific biases. For example, if a class is consistently photographed with a particular background or lighting condition, the model may learn shortcuts that do not generalize to field images. This risk is reduced by augmentation but not fully eliminated. A more robust system would include field images captured under different lighting, camera angles, backgrounds, and disease stages.

9.3 Confidence Scores

The Gradio demo output sometimes shows extremely high confidence, such as 100% for a correct corn common rust sample. High confidence can be useful for user interpretation, but it should not be treated as medical or agricultural certainty. Neural network confidence is not always calibrated. Future work can include temperature scaling or confidence calibration to make predicted probabilities more reliable.

9.4 Training vs. Validation Accuracy Behavior

The training accuracy is lower than validation accuracy. This behavior is acceptable in this project because training images are heavily augmented and the model is regularized with dropout and label smoothing. Validation images are not augmented and therefore are easier to classify. The loss curves also show stable learning rather than severe overfitting.

10 Application and Demo

10.1 Command-Line Inference

The project includes a command-line inference script, `src/predict.py`. The script loads the trained checkpoint, preprocesses the input image, performs model inference, and prints the top-k predicted classes with confidence scores. This allows the project to be tested without running the web demo.

Example command:

Listing 2: Single-image prediction command.

```
python -m src.predict --image_path path/to/leaf_image.jpg --top_k
3
```

Example output:

Listing 3: Example top-3 prediction output.

```
Prediction Results
-----
1. Corn (maize) - Common rust (100.00%)
2. Corn (maize) - Northern Leaf Blight (0.00%)
3. Orange - Haunglongbing Citrus greening (0.00%)
```

10.2 Gradio Web Demo

A Gradio demo was created to make the system easy to test. The user uploads a plant leaf image, presses submit, and receives the top-3 predicted classes. The demo is useful for project discussion because it demonstrates end-to-end functionality beyond offline testing.



Figure 6: Working Gradio demo predicting Corn (maize) Common rust with top-3 probabilities.

11 Repository Structure and Reproducibility

11.1 Repository Structure

The GitHub repository is structured as follows:

Listing 4: Final repository structure.

```
crop-disease-detection/  
|-- app/  
|   |-- gradio_app.py  
|-- data/  
|   |-- raw/  
|   |-- processed/  
|       |-- class_names.json  
|       |-- class_to_idx.json  
|-- models/  
|   |-- best_model.pth  
|-- outputs/  
|   |-- accuracy_curve.png  
|   |-- loss_curve.png  
|   |-- confusion_matrix.png
```

```
|    |-- random_sample_predictions.png
|    |-- test_results.json
|    |-- classification_report.txt
|-- src/
|    |-- dataset.py
|    |-- model.py
|    |-- train.py
|    |-- evaluate.py
|    |-- predict.py
|    |-- sample_predictions.py
|-- requirements.txt
|-- README.md
|-- .gitignore
```

11.2 Reproducible Instructions

To reproduce the project, a user should clone the repository, install requirements, download the dataset, organize the data, train the model, and run evaluation. The README file contains these instructions in detail. A simplified workflow is:

Listing 5: Reproducible project workflow.

```
git clone https://github.com/ziadabdelhafizz/crop-disease-
detection.git
cd crop-disease-detection
pip install -r requirements.txt
python -m src.train --epochs 5 --batch_size 32 --lr 0.001 --
    patience 3
python -m src.evaluate --batch_size 32
python app/gradio_app.py
```

11.3 Role of Each Source File

Table 5: Main source files and their responsibilities.

File	Responsibility
<code>src/dataset.py</code>	Defines preprocessing, augmentation, and DataLoader creation.
<code>src/model.py</code>	Builds the EfficientNet-B0 transfer learning model.
<code>src/train.py</code>	Trains the model, saves the best checkpoint, and plots training curves.
<code>src/evaluate.py</code>	Evaluates the model on the test set and saves metrics, confusion matrix, and predictions.
<code>src/predict.py</code>	Runs command-line inference on a single image.
<code>src/sample_predictions.py</code>	Generates visual prediction examples for documentation.
<code>app/gradio_app.py</code>	Provides a browser-based user interface for image upload and prediction.

12 Limitations

The project has several limitations. First, the dataset consists mainly of isolated leaf images, so the model may not generalize perfectly to real field images. Second, the system performs classification only; it does not localize the diseased region or estimate severity. Third, some classes are visually similar, making misclassification possible even when the model performs well overall. Fourth, the model’s confidence scores may be overconfident because no calibration method was applied. Finally, the system does not provide treatment recommendations, because that would require agricultural expertise and should not be generated automatically without validation.

13 Future Work

Future improvements can make the project more practical and innovative:

1. Add Grad-CAM heatmaps to explain which leaf regions influenced the prediction.
2. Train and evaluate on real field images with complex backgrounds.
3. Add leaf segmentation before classification to reduce background bias.

4. Fine-tune the full EfficientNet backbone for more epochs and compare with ResNet50 and MobileNetV3.
5. Add confidence calibration to make probability outputs more reliable.
6. Deploy the model using Hugging Face Spaces, Streamlit Cloud, or a mobile application.
7. Add a disease information page that explains symptoms and suggests consulting an agricultural expert.

14 Conclusion

This project successfully developed a functional crop disease detection system using computer vision and deep learning. The system uses EfficientNet-B0 transfer learning to classify plant leaf images into 38 crop-disease categories. The pipeline includes dataset setup, preprocessing, augmentation, model training, evaluation, single-image prediction, a Gradio demo, and a documented GitHub repository. The final model achieved 94.59% test accuracy, 94.70% weighted precision, 94.59% weighted recall, and 94.58% weighted F1-score on 8,795 test images. These results demonstrate that transfer learning is effective for plant disease classification under controlled image conditions.

The project also includes error analysis showing that most mistakes occur between visually similar diseases. This provides a realistic understanding of model behavior and helps identify future improvements. Overall, the system satisfies the requirements of a university computer vision project by combining accurate model development, clear documentation, reproducible code, and a working application demo.

A Appendix A: Full Class List

The complete 38-class dataset includes combinations of crop species and health conditions. The classes are:

1. Apple - Apple scab
2. Apple - Black rot
3. Apple - Cedar apple rust
4. Apple - healthy
5. Blueberry - healthy
6. Cherry including sour - Powdery mildew

7. Cherry including sour - healthy
8. Corn maize - Cercospora leaf spot / Gray leaf spot
9. Corn maize - Common rust
10. Corn maize - Northern Leaf Blight
11. Corn maize - healthy
12. Grape - Black rot
13. Grape - Esca / Black Measles
14. Grape - Leaf blight / Isariopsis Leaf Spot
15. Grape - healthy
16. Orange - Haunglongbing / Citrus greening
17. Peach - Bacterial spot
18. Peach - healthy
19. Pepper bell - Bacterial spot
20. Pepper bell - healthy
21. Potato - Early blight
22. Potato - Late blight
23. Potato - healthy
24. Raspberry - healthy
25. Soybean - healthy
26. Squash - Powdery mildew
27. Strawberry - Leaf scorch
28. Strawberry - healthy
29. Tomato - Bacterial spot
30. Tomato - Early blight
31. Tomato - Late blight

- 32. Tomato - Leaf Mold
- 33. Tomato - Septoria leaf spot
- 34. Tomato - Spider mites / Two-spotted spider mite
- 35. Tomato - Target Spot
- 36. Tomato - Tomato Yellow Leaf Curl Virus
- 37. Tomato - Tomato mosaic virus
- 38. Tomato - healthy

B Appendix B: Key Implementation Choices

B.1 Why the Dataset Was Not Pushed Fully to GitHub

The dataset contains tens of thousands of images and is too large for a normal GitHub repository. Therefore, the repository includes source code, trained model, outputs, and instructions, but excludes raw and processed image folders using `.gitignore`. This is a standard reproducibility practice. Users can download the dataset from Kaggle and recreate the folder structure locally or in Colab.

B.2 Why Top-3 Predictions Were Added

Top-3 predictions improve interpretability. Plant diseases can have similar visual symptoms, so showing only the top-1 class hides uncertainty. A top-3 output allows users to see whether the model is considering related diseases. This is especially useful for tomato and corn classes, where the confusion matrix shows several visually similar disease pairs.

B.3 Why Gradio Was Used

Gradio was selected because it provides a simple way to convert a trained model into an interactive web demo. It is suitable for academic projects because it can run in Colab and generate a public URL. This allows instructors and classmates to test the model without installing a full web application stack.

C Appendix C: Important Commands

C.1 Training Command

```
python -m src.train --epochs 5 --batch_size 32 --lr 0.001 --  
    patience 3
```

C.2 Evaluation Command

```
python -m src.evaluate --batch_size 32
```

C.3 Prediction Command

```
python -m src.predict --image_path path/to/image.jpg --top_k 3
```

C.4 Demo Command

```
python app/gradio_app.py
```

References

- [1] D. P. Hughes and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” arXiv preprint arXiv:1511.08060, 2015.
- [2] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.
- [3] J. Deng, W. Dong, R. Socher, L. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.
- [4] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proceedings of the 36th International Conference on Machine Learning*, 2019, pp. 6105–6114.
- [5] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 618–626.
- [6] Kaggle, “New Plant Diseases Dataset,” available at: <https://www.kaggle.com/datasets/vipooooool/new-plant-diseases-dataset>.
- [7] TensorFlow Datasets, “PlantVillage dataset catalog entry,” available at: https://www.tensorflow.org/datasets/catalog/plant_village.
- [8] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” arXiv preprint arXiv:1412.6980, 2014.
- [9] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.